Embodied AI Simulation and Reinforcement Learning: From Physics Engines to Sim-to-Real

Making sense of Isaac Sim / Isaac Lab, MuJoCo / mjlab, Genesis World, and Newton

2026-10-30

Contents

1. Understand the stack before choosing a platform . . . 2
2. Understand the robot model: why does it move? . . . 3
3. Understand the RL environment: how does the policy interact? . . . 4
4. A unified RL workflow across frameworks . . . 5
5. Move from simulation to reality: why isn't training enough? . . . 6
6. Common failures and practical principles . . . 6
References and further learning . . . 7

Course home ·简体中文 ·PDF

Item	Description
Purpose	Understand simulators, physics backends, RL frameworks and real-robot deployment.
Running examples	Franka Panda (a basic robot arm) and BrainCo Revo3 (a dexterous hand).
Prerequisites	Introductory robot kinematics, feedback control, and reinforcement learning.
Version policy	Features vary across releases and tasks; this course does not claim a universal speed or accuracy ranking.
License	CC BY-NC-SA 4.0 ·Xiong Zeng and contributors

1. Understand the stack before choosing a platform

1.1 The robot-learning stack in one picture

No single tool performs every step of robot RL. **Model descriptions** specify robot geometry and dynamics. **Physics engines** compute motion and contacts. **Simulation platforms** manage scenes and sensors. **RL frameworks** organize tasks, parallel experience collection, and trainer interfaces. Projects combine several layers; keep their responsibilities explicit.

Layer	Primary responsibility	Examples
Robot and scene description	Bodies, joints, inertia, collisions, and sensors	URDF, MJCF, USD
Physics solvers	Dynamics, constraints, contacts, and integration	PhysX, MuJoCo, Newton solvers, Genesis solvers
Scenes and sensors	Visualization, cameras, scene construction, timing	Isaac Sim, Genesis World, MuJoCo simulation API
RL task framework	Observations, actions, rewards, termination, resets, vectorization	Isaac Lab, mjlab, MuJoCo Playground, custom Genesis environments
Learning and deployment	PPO/SAC optimization, evaluation, safety integration	RSL-RL and other trainers, real-robot controllers

Representative stack combinations (illustrative, not exclusive):

- **Isaac stack**: Isaac Sim + PhysX ⟶ Isaac Lab ⟶ RL trainer, often for scene-heavy or visual learning.
- **Newton stack**: Newton (e.g., MJWarp solver) ⟶ a supported Isaac Lab task ⟶ RL trainer; supported Newton configurations can run without Isaac Sim.
- **MuJoCo / mjlab stack**: MJCF ⟶ MuJoCo Warp ⟶ mjlab ⟶ RL trainer, with native model/data access and modular tasks.
- **Genesis stack**: URDF/MJCF ⟶ Genesis World (physics + scenes + sensors) ⟶ custom or compatible RL environment and trainer.

1.2 What are Genesis, Newton, and mjlab?

Platform or component	Primary role	Key limitation or distinction
Isaac Sim	USD assets, scenes, physics, cameras, rendering	Not an RL algorithm; often paired with Isaac Lab.
Isaac Lab	Robot-learning tasks and environment framework	Not synonymous with PhysX; backend support depends on the task.
MuJoCo	Rigid-body, joints, actuators, contacts	Not itself an RL task framework; distinguish CPU and batched GPU paths.
MJX / MuJoCo Warp	Batched implementations in the MuJoCo ecosystem	MJX-JAX and MJWarp differ in computation and differentiability.
mjlab	Lightweight GPU RL framework built on MuJoCo Warp	Manager-based design inspired by Isaac Lab; high-fidelity RGB is not its focus.
Genesis World (formerly Genesis)	Pythonic, multi-physics simulation platform	Combines rigid, deformable, and particle solvers; verify individual features.
Newton	Extensible Warp-based GPU physics engine	Several solvers; not a substitute for a complete RL task framework.
MuJoCo Playground	Ready-made MuJoCo robot-learning tasks	Useful for benchmarks; distinct from the MuJoCo engine.

Two especially important distinctions. First, **Newton is not simply another name for MJWarp**. Newton can expose MuJoCo Warp through its solver interface alongside other solvers; differentiability is solver-specific and is not a property of every MJWarp rollout. Second, **mjlab directly uses MuJoCo Warp**. That does not imply automatic access to Newton's other solvers. Verify these relationships against the versions actually installed.

1.3 Choose by task, not marketing claims

Research need	Start by considering	What really matters
Robot mechanics and control debugging	MuJoCo	Transparent actuators, models, and contacts
Lightweight structured, parallel RL	mjlab + MuJoCo Warp	Rigid-body tasks, PyTorch tensors, composable MDP terms
Multi-camera visual tasks and complex scenes	Isaac Sim + Isaac Lab	Rendering, sensors, and task compatibility
Soft bodies, particles, and coupled materials	Genesis World or a suitable Newton solver	Material model, coupling, numerical behavior
Physics-backend comparisons	Isaac Lab with supported Newton / PhysX choices	Task support, interface alignment, solver validation

A GPU-accelerated or differentiable platform is **not automatically faster, more accurate, or easier to transfer** for dexterous manipulation. We use **Panda grasping a cube** to develop intuition and **Revo3 grasping an object** to discuss actuator coupling, inertia, and fingertip contacts. Neither is a hands-on exercise.

2. Understand the robot model: why does it move?

2.1 One robot, three common descriptions

Format	Primary use	What may change during conversion
URDF	Links, joints, geometry, and basic inertial properties	Actuator details, contact solver settings, scene configuration
MJCF	MuJoCo models, actuators, constraints, and contacts	Solver-specific behavior may lack an exact counterpart elsewhere
USD	Isaac scene graphs, assets, physics, and rendering properties	Imported drives, collision shapes, and sensors need validation

Successful import does **not** imply physical equivalence. Maintain a **single source of truth for robot parameters**: joint names, axes, zero positions, limits, masses, centers of mass, inertia tensors, collision shapes, actuator types, tool frames, and sensor extrinsics.

2.2 Six layers: from geometry to contact

1. Kinematics. How are joints connected? What are their axes, zero positions, and limits? For Panda, verify the arm and tool-center point (TCP). For Revo3, also inspect joint order, any coupling, and each finger's range of motion.

2. Mass and inertia. Link mass, center of mass, and rotational inertia determine gravitational and inertial response. A visually convincing model can still have incorrect grasping dynamics.

3. Actuation. After the policy specifies a joint target, how does the low-level servo generate effort? Specify control mode, proportional gain K_p, derivative gain K_d, speed/effort limits, and response latency. **Armature** represents additional effective inertia at a joint, often associated with motor and transmission dynamics. It is not a parameter that should simply

be maximized, and it does not replace identifying the real mechanism. Different finger joints may use different values, provided those choices are justified by the model or measurements.

4. **Collision and contact**. Visual meshes are not necessarily collision shapes. Whether a fingertip can hold an object depends on collision geometry, friction, the numerical solver, and actuator output. Increasing friction alone rarely repairs penetration or oscillation.

5. **Sensors**. For encoders, cameras, force sensors, and tactile devices, define units, reference frames, sample timestamps, and noise. Ground-truth object poses available in simulation are not necessarily valid observations for a deployable policy.

6. **Constraints and coupling**. If the Revo3 asset being used includes linked joints, transmissions, or tendon constraints, validate these against the actual mechanism. Do not assume every visible joint can receive independent torque commands.

2.3 A target position is not a teleport command

A simple position servo can be represented conceptually by

$$\tau = \mathrm{sat}\Big(K_p(q_{\mathrm{target}} - q) - K_d\dot{q}\Big),$$

where sat enforces effort limits. A real robot may also include gravity compensation, friction, filtering, and additional internal control loops. When the **action is a target position**, the controller and dynamics still determine motion. Writing the joint state directly to that target may bypass physics and produce misleading sim-to-real results.

Keep three clocks separate: **physics integration**, **low-level control, and policy/camera sampling**. For example, at 500 Hz physics, 250 Hz control, and 50 Hz policy inference, each policy decision spans 10 physics steps and five controller updates. A 30 Hz camera does not provide a synchronized new image at every policy step. These numbers illustrate timing only; **they are not Revo3 specifications**.

Takeaway: Before comparing simulators, align the robot structure, actuator semantics, action scales, integration periods, and contact settings as closely as possible.

Cross-engine model checks: Importing one URDF, MJCF, or USD into Isaac Sim, MuJoCo, Genesis World, or Newton does not ensure identical collisions, actuators, constraints, friction, or sensors. Validate joints and static contacts first.

3. Understand the RL environment: how does the policy interact?

An RL environment wraps continuous robot simulation into a trainable loop: **read observations → choose a policy action → execute the controller → advance physics → compute reward and completion conditions → reset when required**.

Interface	Plain-language meaning	Conceptual Revo3 grasping example
Observation	What the policy can see	Finger positions/velocities; deployable object-pose estimates or tactile data
Action	What the policy asks the robot to do	Bounded joint-position targets or increments
Reward	What behavior is encouraged	Approach, useful contact, stable holding; penalties for unsafe behavior
Termination	Whether an episode truly ends	Success or unrecoverable failure; time limits tracked separately
Reset	Where the next episode begins	Restore finger states, reposition the object, clear history buffers

Observations are not the entire simulator state. During training, a critic may use privileged object state. But if the deployed actor only has joints and cameras, it must not silently depend on the object's ground-truth pose. If tactile feedback is required, verify that a real sensor exists and model its timing.

Actions are not always torques. Two environments can both expose a 20-dimensional action vector while one means joint-position increments and the other means torque commands. Align joint order, semantics, scaling, clipping, and update period before comparing policies.

Termination is not truncation. An irrecoverable drop and reaching the episode time limit are different events. They should be tracked separately for sound learning targets and success statistics. Reset must synchronize physical state, controller memory, and observation history.

Frameworks only implement these concepts. In Isaac Lab, mjlab, Playground, or a custom Genesis environment, the five RL interfaces mean the same thing. Equivalent interface names do not guarantee matching observation timing, action scaling, rewards, or resets.

4. A unified RL workflow across frameworks

4.1 Eight steps shared by every stack

Validate the robot model → define the five RL interfaces → inspect actions and contacts → establish a simple baseline → scale up training → evaluate independently → add plausible randomization → archive reproducible settings. This is our only training workflow; there is no need to memorize separate recipes for Isaac Lab, mjlab, and Genesis.

Validate a fixed, interpretable starting condition before scaling out or adding complex contacts. If reward rises but grasp success does not, check success criteria, reward loopholes, and observation design before enlarging the policy network.

4.2 How engineering entry points differ

Stack	Scene and model	RL organization	First things to verify
Isaac Lab (PhysX)	Isaac Sim, USD, articulations	Manager-based / Direct vectorized tasks	Rendering, sensors, assets, PhysX settings
Isaac Lab (Newton)	Warp-native backend; task-selected solver	Supported tasks under the shared environment API	Solver support, feature coverage, data and joint ordering
mjlab / MuJoCo Warp	MJCF, `MjModel`, `MjData`	Manager-based style with GPU rollout	Actuators, contacts, model data, GPU requirements
MuJoCo Playground	MuJoCo/MJX ecosystem models	Task library, wrappers, compatible training scripts	Selected MJX implementation, task/trainer interface
Genesis World	Python scene with multi-physics solvers	Custom or compatible RL environment	Material models, coupled contacts, observations, trainer wiring

Manager-based and Direct are two ways to implement Isaac Lab tasks, not distinct RL theories. mjlab uses reusable manager-based MDP terms but keeps native MuJoCo data access. Genesis World integrates physics and scenes, while the learning algorithm still needs a suitable environment/trainer interface. Newton belongs at the physics layer rather than acting as another PPO implementation.

4.3 Where do PPO, SAC, and differentiable simulation fit?

PPO is widely used for synchronous, batched rollouts; **SAC** learns off-policy with a replay buffer. Physics engines do not automatically guarantee support for SAC, GPU rollout, or a particular observation pipeline. Verify trainer integration.

Differentiable simulation means gradients can propagate through certain solver operations; it may help with system identification or gradient-based optimization, but is **not required** for standard PPO or SAC. Newton and Genesis capabilities are solver-specific. In the MuJoCo ecosystem, MJX-JAX and MJWarp should not be conflated when discussing autodiff.

4.4 Measure more than peak FPS

Report **environment steps/s, physics steps/s, policy update time, peak memory, and wall-clock time to the target success rate**. For visual or dexterous tasks, separately profile rendering, encoding, contact solving, and CPU–GPU transfers. Fair comparisons align mass/inertia, action semantics, control rates, disturbances, success criteria, and training budgets.

Replaying the same policy in multiple engines can reveal model sensitivity, but differences may arise from assets, actuation, contact, and timing—not just from one engine being more physically accurate.

5. Move from simulation to reality: why isn't training enough?

5.1 Five sources of the sim-to-real gap

Source	What to inspect for Panda / Revo3
Geometry and inertia	Joint zero positions, TCP, link inertia, payload mass
Actuation	Position/current/torque semantics, K_p/K_d, saturation, delay
Contacts	Fingertip collision shapes, friction, slip, compliance
Observations	Camera calibration, depth error, occlusion, tactile noise
Timing and communications	Control rates, timestamps, buffering, packet loss, action filtering

System identification estimates the important parameters of the real mechanism and objects; **domain randomization** then trains robustness to plausible remaining uncertainty. Randomization does not correct a fundamentally wrong robot model, and wider ranges are not always more useful.

A reasonable progression is **learn under fixed conditions → vary initial poses → randomize dynamics and actuators → add state-estimation/visual errors → add disturbances and recovery**. For Revo3 grasping, object mass and friction, finger response, fingertip contact, and slip often deserve attention before elaborate visual background variation.

5.2 Preserve the interface contract on the real robot

At minimum, check that **joint order and reference frames** agree; **action meanings and limits** match; **policy, controller, and sensor frequencies and latency** are compatible; **observations are truly available on the robot**; and independent **safety constraints** enforce speed, effort, workspace, and emergency stop behavior.

A state-based teacher can be useful during training, but the policy deployed after vision distillation must be the student that can obtain its inputs from real sensors—not a teacher that requires simulator-only ground truth. Also check quaternion component order (`wxyz` versus `xyzw`) when transferring poses.

5.3 Evaluate robustness, not only reward

Report at least **success rate on held-out starts, time to success, slip/drop rate, recovery rate after perturbations, and unsafe-event rate**, together with simulation interactions and wall-clock training time. Multiple training seeds and held-out disturbances help distinguish training-distribution fit from generalization. Testing across PhysX, MuJoCo, Genesis, or Newton solvers can expose model dependence but cannot guarantee real-world success.

6. Common failures and practical principles

Symptom	Check first	Avoid doing immediately
Robot shakes or joints diverge	Zero positions, axes, K_p/K_d, timestep, effort limits	Blindly raising gains
Fingertips penetrate or cannot hold	Collision shapes, contact settings, friction, servo response	Only raising friction

Symptom	Check first	Avoid doing immediately
Reward improves but grasping fails	Success checks, reward loopholes, termination/reset	Only extending training
Training is slow or runs out of memory	Environment count, cameras, rendering, transfers	Maximizing environment count
Simulation works but hardware fails	Joint/frame mapping, action semantics, latency, friction, calibration	Immediately enlarging the policy

Unified debugging order: robot model → units and reference frames → actuation → contact and timestep → observation/latency → RL task → optimization algorithm. Tuning PPO or SAC before checking upstream errors often misses the actual problem.

Course summary: MuJoCo makes mechanics and control inspectable; Isaac Sim supports scenes and vision; Isaac Lab and mjlab structure RL tasks; Genesis World extends multi-physics simulation; Newton offers extensible GPU physics solvers. There is **no universal best platform**. Correct models, explicit interfaces, reproducible evaluation, and safe real-robot behavior matter more than a framework name.

References and further learning

1. Isaac Sim documentation — scenes, USD, rendering, sensors.
2. Isaac Lab documentation and physics backends — RL environments, supported Newton/PhysX configurations.
3. MuJoCo documentation — dynamics, actuators, contact.
4. MuJoCo MJX documentation — MJX-JAX versus MJX-Warp.
5. mjlab documentation — MuJoCo Warp and manager-based RL environments.
6. MuJoCo Playground — robot-learning tasks and benchmarks.
7. Genesis World documentation — multi-physics solvers, scenes, sensors.
8. Newton documentation and solver comparison — Warp engine and solver-specific capabilities.
9. MuJoCo Menagerie — public robot models.
10. BrainCo robot descriptions — validate the exact Revo3 asset revision.